

EXPERIMENT -3

AIM

To create an Employee–Department–Project relational database, insert at least 30 employees across 5 departments and 8 projects, and demonstrate SQL queries using Selection, Projection, Aggregate Functions, GROUP BY, HAVING, CASE expressions, and ORDER BY.

SOURCE CODE

```
CREATE DATABASE CompanyDB;
```

```
USE CompanyDB;
```

```
CREATE TABLE Department (
```

```
    dept_id INT PRIMARY KEY,
```

```
    dept_name VARCHAR(50) NOT NULL
```

```
);
```

```
CREATE TABLE Project (
```

```
    project_id INT PRIMARY KEY,
```

```
    project_name VARCHAR(100) NOT NULL,
```

```
    budget DECIMAL(12,2),
```

```
    dept_id INT,
```

```
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
```

```
);
```

```
CREATE TABLE Employee (
```

```
    emp_id INT PRIMARY KEY,
```

```
    emp_name VARCHAR(50) NOT NULL,
```

```
    salary DECIMAL(10,2),
```

```
    hire_date DATE,
```

```
    dept_id INT,
```

```
    project_id INT,
```

```
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id),
```

```
    FOREIGN KEY (project_id) REFERENCES Project(project_id)
```

```
);
```

Insert Departments

INSERT INTO Department VALUES

(1,'IT'),
(2,'HR'),
(3,'Finance'),
(4,'Marketing'),
(5,'Operations');

Insert Projects

INSERT INTO Project VALUES

(101,'Cloud Migration',150000,1),
(102,'AI Analytics',200000,1),
(103,'Recruitment Portal',80000,2),
(104,'Financial Dashboard',120000,3),
(105,'Digital Campaign',90000,4),
(106,'Supply Chain System',180000,5),
(107,'Mobile Application',140000,1),
(108,'Employee Wellness',60000,2);

Insert 30 Employees

INSERT INTO Employee VALUES

(1,'Ujjawal',650000,'2022-01-10',1,101),
(2,'Krish',72000,'2021-03-15',1,102),
(3,'Umesh',58000,'2023-06-20',1,107),
(4,'Jonny',81000,'2020-08-12',1,101),
(5,'karina',69000,'2022-11-05',1,102),
(6,'Reyansh',62000,'2024-02-18',1,107),
(7,'Ananya',52000,'2022-04-11',2,103),
(8,'Diya',48000,'2023-01-22',2,108),
(9,'Myra',61000,'2021-07-19',2,103),
(10,'Sara',55000,'2022-09-30',2,108),
(11,'Ishita',47000,'2024-03-14',2,103),
(12,'Meera',59000,'2020-12-01',2,108),

(13, 'Rohan', 75000, '2021-02-17', 3, 104),
(14, 'Karan', 68000, '2022-05-09', 3, 104),
(15, 'Nikhil', 83000, '2019-10-21', 3, 104),
(16, 'Yash', 62000, '2023-08-13', 3, 104),
(17, 'Manav', 71000, '2021-11-28', 3, 104),
(18, 'Dev', 56000, '2024-01-09', 3, 104),
(19, 'Aanya', 54000, '2022-02-25', 4, 105),
(20, 'Kiara', 63000, '2021-06-16', 4, 105),
(21, 'Tanya', 57000, '2023-04-10', 4, 105),
(22, 'Riya', 66000, '2020-09-07', 4, 105),
(23, 'Avni', 51000, '2024-02-05', 4, 105),
(24, 'Navya', 70000, '2022-12-19', 4, 105),
(25, 'Samar', 60000, '2021-01-12', 5, 106),
(26, 'Dhruv', 73000, '2020-04-23', 5, 106),
(27, 'Atharv', 67000, '2022-07-18', 5, 106),
(28, 'Parth', 59000, '2023-05-29', 5, 106),
(29, 'Rudra', 76000, '2019-08-31', 5, 106),
(30, 'Veer', 64000, '2024-01-20', 5, 106);

SQL QUERIES

1. Selection

Display employees whose salary is greater than 70,000.

*SELECT **

FROM Employee

WHERE salary > 70000;

OUTPUT

 AVINASH JHA

I/O	AI Agent	107 ms 
-----	----------	--

STDIN

Input for the program (Optional)

Output:

emp_id	emp_name	salary	hire_date	dept_id	project_id
1	Ujjawal	65000.00	2022-01-10	1	101
2	Krish	72000.00	2021-03-15	1	102
4	Jonny	81000.00	2020-08-12	1	101
13	Rohan	75000.00	2021-02-17	3	104
15	Nikhil	83000.00	2019-10-21	3	104
17	Manav	71000.00	2021-11-28	3	104
26	Dhruv	73000.00	2020-04-23	5	106
29	Rudra	76000.00	2019-08-31	5	106

2. Projection

Display only employee names and salaries.

SELECT emp_name, salary

FROM Employee;

OUTPUT

 AVINASH JHA

I/O	AI Agent	67 ms 
-----	----------	---

STDIN

Input for the program (Optional)

Output:

emp_name	salary
Ujjawal	65000.00
Krish	72000.00
Umesh	58000.00
Jonny	81000.00
karina	69000.00
Reyansh	62000.00
Ananya	52000.00
Diya	48000.00
Myra	61000.00
Sara	55000.00
Ishita	47000.00
Meera	59000.00
Rohan	75000.00
Karan	68000.00
Nikhil	83000.00
Yash	62000.00
Manav	71000.00
Dev	56000.00
Aanya	54000.00
Kiara	63000.00
Tanya	57000.00
Riya	66000.00

3. Aggregate Functions

Find total employees, average salary, maximum salary, minimum salary and total salary.

SELECT

COUNT() AS total_employees,*

AVG(salary) AS average_salary,

MAX(salary) AS highest_salary,

MIN(salary) AS lowest_salary,

SUM(salary) AS total_salary

FROM Employee;

 AVINASH JHA

 I/O  AI Agent 90 ms 

STDIN

Input for the program (Optional)

Output:

total_employees	average_salary	highest_salary	lowest_salary	total_salary
30	82966.666667	650000.00	47000.00	2489000.00

4. GROUP BY

Display the number of employees and average salary in each department.

SELECT

d.dept_name,

COUNT(e.emp_id) AS employee_count,

AVG(e.salary) AS average_salary

FROM Department d

JOIN Employee e

ON d.dept_id = e.dept_id

GROUP BY d.dept_id, d.dept_name;

OUTPUT

 AVINASH JHA

I/O AI Agent 81 ms 

STDIN

Input for the program (Optional)

Output:

dept_name	employee_count	average_salary
IT	6	165333.333333
HR	6	53666.666667
Finance	6	69166.666667
Marketing	6	60166.666667
Operations	6	66500.000000

5. HAVING

Display departments whose average salary is greater than 65,000.

SELECT

d.dept_name,

AVG(e.salary) AS average_salary

FROM Department d

JOIN Employee e

ON d.dept_id = e.dept_id

GROUP BY d.dept_id, d.dept_name

HAVING AVG(e.salary) > 65000;

OUTPUT

 AVINASH JHA

I/O AI Agent 90 ms 

STDIN

Input for the program (Optional)

Output:

dept_name	average_salary
IT	165333.333333
Finance	69166.666667
Operations	66500.000000

6. CASE Expression

Classify employees according to their salary.

SELECT

emp_name,

salary,

CASE

WHEN salary >= 75000 THEN 'High Salary'

WHEN salary >= 60000 THEN 'Medium Salary'

ELSE 'Low Salary'

END AS salary_category

FROM Employee;

OUTPUT



AVINASH JHA

I/O

AI Agent

100 ms

STDIN

Input for the program (Optional)

Output:

emp_name	salary	salary_category
Ujjawal	650000.00	High Salary
Krish	72000.00	Medium Salary
Umesh	58000.00	Low Salary
Jonny	81000.00	High Salary
karina	69000.00	Medium Salary
Reyansh	62000.00	Medium Salary
Ananya	52000.00	Low Salary
Diya	48000.00	Low Salary
Myra	61000.00	Medium Salary
Sara	55000.00	Low Salary
Ishita	47000.00	Low Salary
Meera	59000.00	Low Salary
Rohan	75000.00	High Salary
Karan	68000.00	Medium Salary
Nikhil	83000.00	High Salary
Yash	62000.00	Medium Salary
Manav	71000.00	Medium Salary
Dev	56000.00	Low Salary
Aanya	54000.00	Low Salary
Kiara	63000.00	Medium Salary
Tanya	57000.00	Low Salary
Riya	66000.00	Medium Salary

7. ORDER BY

Display employees in descending order of salary.

```
SELECT emp_name, salary
```

```
FROM Employee
```

```
ORDER BY salary DESC;
```

OUTPUT

 AVINASH JHA

I/O AI Agent 86 ms 

STDIN

Input for the program (Optional)

Output:

emp_name	salary
Nikhil	83000.00
Arjun	81000.00
Rudra	76000.00
Rohan	75000.00
Dhruv	73000.00
Vivaan	72000.00
Manav	71000.00
Navya	70000.00
Kabir	69000.00
Karan	68000.00
Atharv	67000.00
Riya	66000.00
Aarav	65000.00
Veer	64000.00
Kiara	63000.00

8. JOIN Employee, Department and Project

Display employee name, department, project and salary.

```
SELECT
```

```
e.emp_name,
```

```
d.dept_name,
```

```
p.project_name,
```

```
e.salary
```

```
FROM Employee e
```

```
JOIN Department d
```

ON e.dept_id = d.dept_id

JOIN Project p

ON e.project_id = p.project_id

ORDER BY d.dept_name, e.emp_name;

OUTPUT

 AVINASH JHA

I/O	AI Agent	83 ms	✓
STDIN			
Input for the program (Optional)			
Output:			
emp_name	dept_name	project_name	salary
Dev	Finance	Financial Dashboard	56000.00
Karan	Finance	Financial Dashboard	68000.00
Manav	Finance	Financial Dashboard	71000.00
Nikhil	Finance	Financial Dashboard	83000.00
Rohan	Finance	Financial Dashboard	75000.00
Yash	Finance	Financial Dashboard	62000.00
Ananya	HR	Recruitment Portal	52000.00
Diya	HR	Employee Wellness	48000.00
Ishita	HR	Recruitment Portal	47000.00
Meera	HR	Employee Wellness	59000.00
Myra	HR	Recruitment Portal	61000.00
Sara	HR	Employee Wellness	55000.00
Aarav	IT	Cloud Migration	65000.00
Aditya	IT	Mobile Application	58000.00
Ariun	IT	Cloud Migration	81000.00

RESULT

The **Employee–Department–Project database** was successfully created with:

- **30 Employees**
- **5 Departments**
- **8 Projects**

The following SQL concepts were successfully demonstrated:

1. Selection
2. Projection
3. Aggregate Functions
4. GROUP BY
5. HAVING
6. CASE Expression
7. ORDER BY

EXPERIMENT - 4

AIM

Using the Employee schema, write and execute SQL queries with INNER JOIN, LEFT JOIN, self-join, 3-way join, correlated subqueries, EXISTS, and simulated INTERSECT and EXCEPT. Compare the execution plans of selected queries using EXPLAIN.

OBJECTIVES

- To understand how different types of JOIN operations combine rows from related tables.
- To perform a self-join on the Employee table to compare employees within the same department.
- To use correlated subqueries and EXISTS for row-by-row and existence-based filtering.
- To simulate INTERSECT and EXCEPT operations using subqueries supported by MySQL.
- To inspect and compare query execution plans using EXPLAIN.

DATABASE USED

The queries use the CompanyDB database and the Employee–Department–Project schema from the reference experiment. The reference document defines 30 employees, 5 departments and 8 projects. The Employee table contains emp_id, emp_name, salary, hire_date, dept_id and project_id, with foreign keys to Department and Project.

USE CompanyDB;

SCHEMA REFERENCE

Table	Important Columns	Relationship
Employee	emp_id, emp_name, salary, hire_date, dept_id, project_id	dept_id → Department; project_id → Project
Department	dept_id, dept_name	Primary key: dept_id
Project	project_id, project_name, budget, dept_id	Primary key: project_id; dept_id → Department

The source document creates Department, Project and Employee in this order and establishes the foreign-key relationships shown above.

SOURCE CODE / SQL QUERIES

1. INNER JOIN

Display employee names, salaries and their department names using an INNER JOIN.

```
SELECT
    e.emp_id,
    e.emp_name,
    e.salary,
    d.dept_name
FROM Employee e
INNER JOIN Department d
    ON e.dept_id = d.dept_id
ORDER BY e.emp_id;
```

Output

 AVINASH JHA

I/O AI Agent 85 ms 

STDIN

Input for the program (Optional)

Output:

emp_id	emp_name	salary	dept_name
1	Aarav	65000.00	IT
2	Vivaan	72000.00	IT
3	Aditya	58000.00	IT
4	Arjun	81000.00	IT
5	Kabir	69000.00	IT
6	Reyansh	62000.00	IT
7	Ananya	52000.00	HR
8	Diya	48000.00	HR
9	Myra	61000.00	HR
10	Sara	55000.00	HR
11	Ishita	47000.00	HR
12	Meera	59000.00	HR
13	Rohan	75000.00	Finance
14	Karan	68000.00	Finance
15	Nikhil	83000.00	Finance
16	Yash	62000.00	Finance
17	Manav	71000.00	Finance
18	Dev	56000.00	Finance
19	Aanya	54000.00	Marketing
20	Kiara	63000.00	Marketing
21	Tanya	57000.00	Marketing
22	Riya	66000.00	Marketing
23	Avni	51000.00	Marketing
24	Navya	70000.00	Marketing
25	Samar	60000.00	Operations
26	Dhruv	73000.00	Operations
27	Atharv	67000.00	Operations
28	Parth	59000.00	Operations
29	Rudra	76000.00	Operations

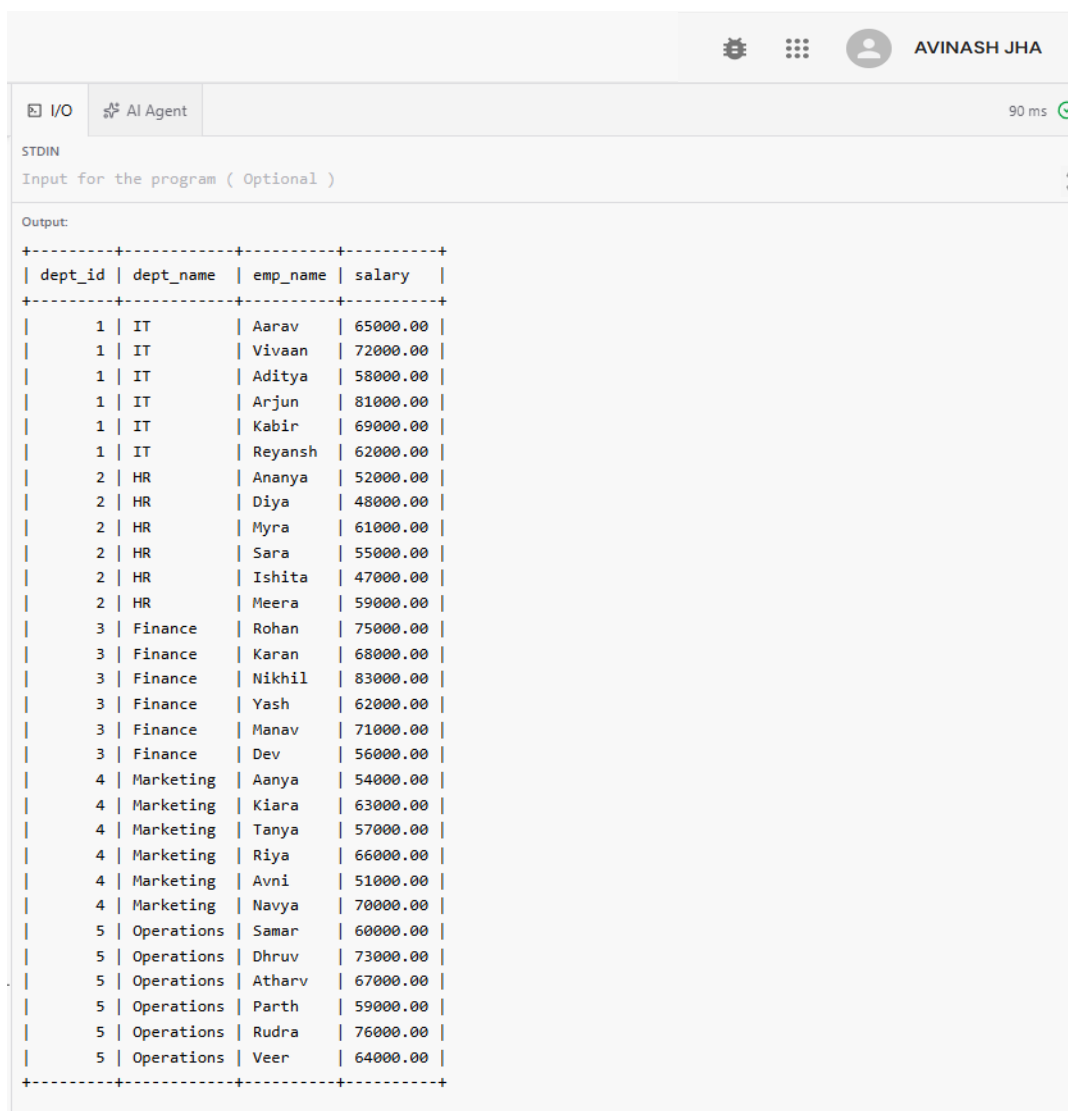
The full query returns all 30 employees. The Department values come from the Department table in the reference data.

2. LEFT JOIN

Display all departments and the employees belonging to them. LEFT JOIN keeps every department from the left table even if no matching employee exists.

```
SELECT
    d.dept_id,
    d.dept_name,
    e.emp_name,
    e.salary
FROM Department d
LEFT JOIN Employee e
    ON d.dept_id = e.dept_id
ORDER BY d.dept_id, e.emp_id;
```

OUTPUT



dept_id	dept_name	emp_name	salary
1	IT	Aarav	65000.00
1	IT	Vivaan	72000.00
1	IT	Aditya	58000.00
1	IT	Arjun	81000.00
1	IT	Kabir	69000.00
1	IT	Reyansh	62000.00
2	HR	Ananya	52000.00
2	HR	Diya	48000.00
2	HR	Myra	61000.00
2	HR	Sara	55000.00
2	HR	Ishita	47000.00
2	HR	Meera	59000.00
3	Finance	Rohan	75000.00
3	Finance	Karan	68000.00
3	Finance	Nikhil	83000.00
3	Finance	Yash	62000.00
3	Finance	Manav	71000.00
3	Finance	Dev	56000.00
4	Marketing	Aanya	54000.00
4	Marketing	Kiara	63000.00
4	Marketing	Tanya	57000.00
4	Marketing	Riya	66000.00
4	Marketing	Avni	51000.00
4	Marketing	Navya	70000.00
5	Operations	Samar	60000.00
5	Operations	Dhruv	73000.00
5	Operations	Atharv	67000.00
5	Operations	Parth	59000.00
5	Operations	Rudra	76000.00
5	Operations	Veer	64000.00

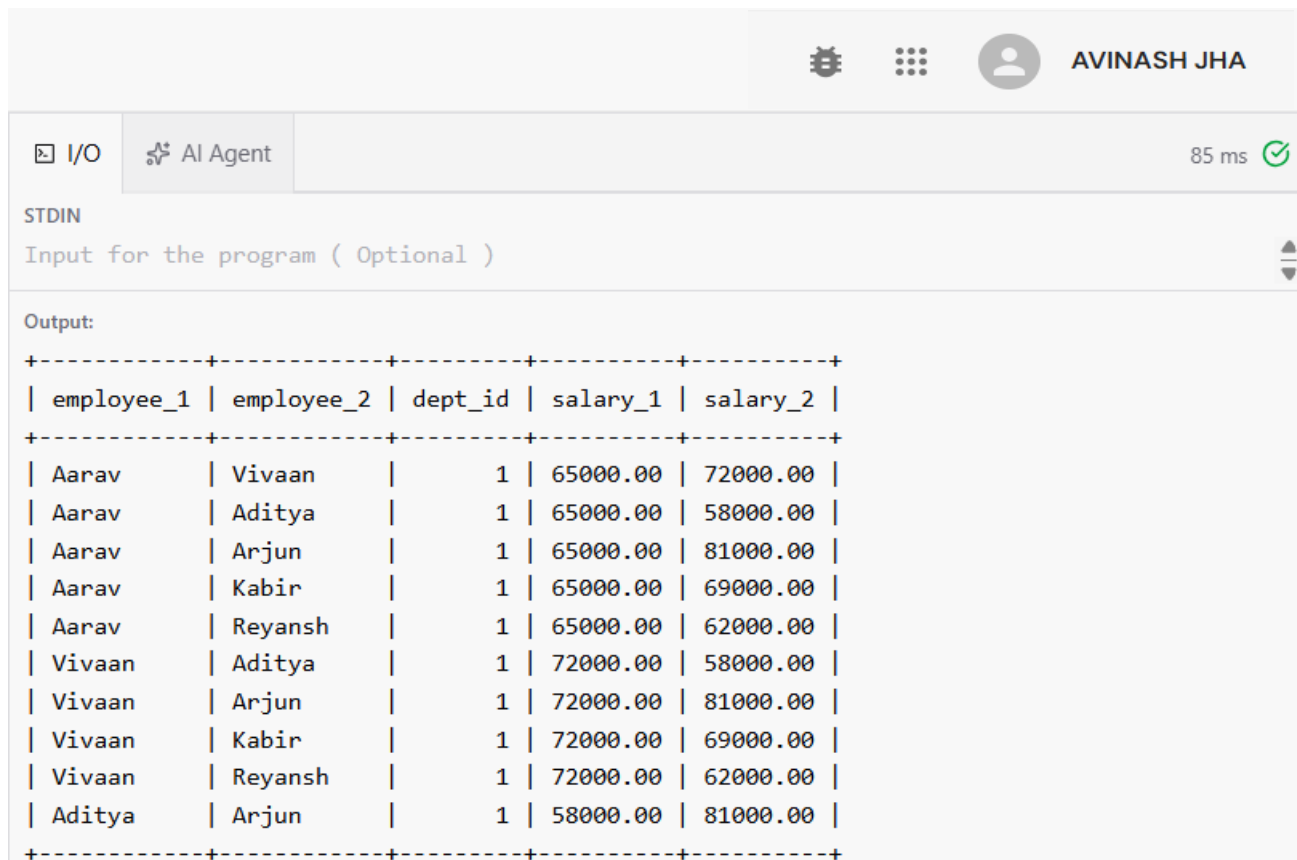
In the supplied data, all five departments have employees, so no NULL employee row appears. The LEFT JOIN would show a department with NULL employee columns if such a department existed.

3. SELF-JOIN

Compare two employees belonging to the same department. The Employee table is joined to itself using two aliases.

```
SELECT
    e1.emp_name AS employee_1,
    e2.emp_name AS employee_2,
    e1.dept_id,
    e1.salary AS salary_1,
    e2.salary AS salary_2
FROM Employee e1
INNER JOIN Employee e2
    ON e1.dept_id = e2.dept_id
    AND e1.emp_id < e2.emp_id
ORDER BY e1.dept_id, e1.emp_id, e2.emp_id
LIMIT 10;
```

OUTPUT (first 10 rows)



employee_1	employee_2	dept_id	salary_1	salary_2
Aarav	Vivaan	1	65000.00	72000.00
Aarav	Aditya	1	65000.00	58000.00
Aarav	Arjun	1	65000.00	81000.00
Aarav	Kabir	1	65000.00	69000.00
Aarav	Reyansh	1	65000.00	62000.00
Vivaan	Aditya	1	72000.00	58000.00
Vivaan	Arjun	1	72000.00	81000.00
Vivaan	Kabir	1	72000.00	69000.00
Vivaan	Reyansh	1	72000.00	62000.00
Aditya	Arjun	1	58000.00	81000.00

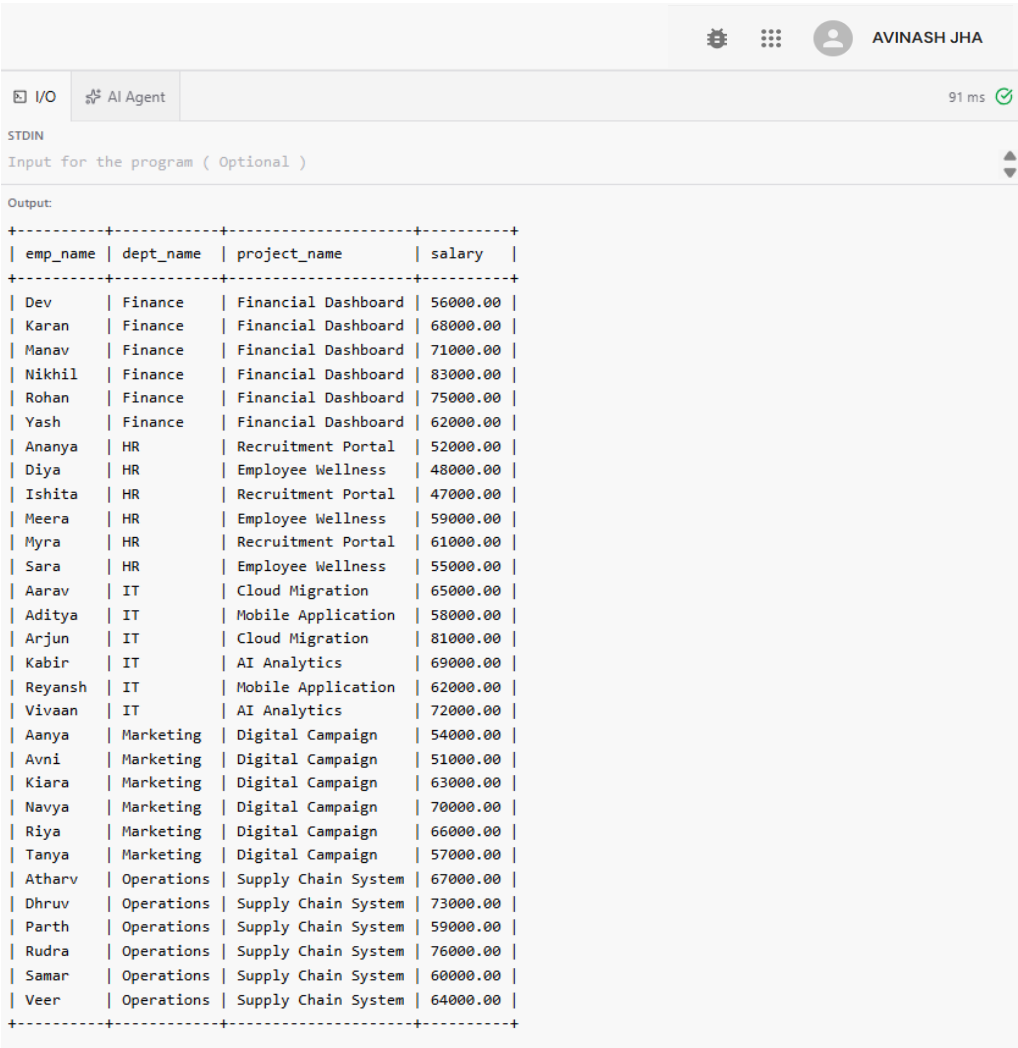
The condition `e1.emp_id < e2.emp_id` prevents the same pair from being repeated in reverse order and prevents an employee from being paired with itself.

4. 3-WAY JOIN

Display employee name, department, project and salary by joining Employee, Department and Project.

```
SELECT
    e.emp_name,
    d.dept_name,
    p.project_name,
    e.salary
FROM Employee e
INNER JOIN Department d
    ON e.dept_id = d.dept_id
INNER JOIN Project p
    ON e.project_id = p.project_id
ORDER BY d.dept_name, e.emp_name;
```

OUTPUT



emp_name	dept_name	project_name	salary
Dev	Finance	Financial Dashboard	56000.00
Karan	Finance	Financial Dashboard	68000.00
Manav	Finance	Financial Dashboard	71000.00
Nikhil	Finance	Financial Dashboard	83000.00
Rohan	Finance	Financial Dashboard	75000.00
Yash	Finance	Financial Dashboard	62000.00
Ananya	HR	Recruitment Portal	52000.00
Diya	HR	Employee Wellness	48000.00
Ishita	HR	Recruitment Portal	47000.00
Meera	HR	Employee Wellness	59000.00
Myra	HR	Recruitment Portal	61000.00
Sara	HR	Employee Wellness	55000.00
Aarav	IT	Cloud Migration	65000.00
Aditya	IT	Mobile Application	58000.00
Arjun	IT	Cloud Migration	81000.00
Kabir	IT	AI Analytics	69000.00
Reyansh	IT	Mobile Application	62000.00
Vivaan	IT	AI Analytics	72000.00
Aanya	Marketing	Digital Campaign	54000.00
Avni	Marketing	Digital Campaign	51000.00
Kiara	Marketing	Digital Campaign	63000.00
Navya	Marketing	Digital Campaign	70000.00
Riya	Marketing	Digital Campaign	66000.00
Tanya	Marketing	Digital Campaign	57000.00
Atharv	Operations	Supply Chain System	67000.00
Dhruv	Operations	Supply Chain System	73000.00
Parth	Operations	Supply Chain System	59000.00
Rudra	Operations	Supply Chain System	76000.00
Samar	Operations	Supply Chain System	60000.00
Veer	Operations	Supply Chain System	64000.00

The reference experiment uses the same three-table join pattern and orders the result by department and employee name.

5. CORRELATED SUBQUERY

Display employees whose salary is greater than the average salary of their own department. The inner query refers to the outer Employee row, making it a correlated subquery.

```
SELECT
    e.emp_id,
    e.emp_name,
    e.salary,
    e.dept_id
FROM Employee e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM Employee e2
    WHERE e2.dept_id = e.dept_id
)
ORDER BY e.dept_id, e.salary DESC;
```

OUTPUT

AVINASH JHA

I/O

AI Agent

86 ms

STDIN

Input for the program (Optional)

Output:

emp_id	emp_name	salary	dept_id
4	Anjun	81000.00	1
2	Vivaan	72000.00	1
5	Kabir	69000.00	1
9	Myra	61000.00	2
12	Meera	59000.00	2
10	Sara	55000.00	2
15	Nikhil	83000.00	3
13	Rohan	75000.00	3
17	Manav	71000.00	3
24	Navya	70000.00	4
22	Riya	66000.00	4
20	Kiara	63000.00	4
29	Rudra	76000.00	5
26	Dhruv	73000.00	5
27	Atharv	67000.00	5

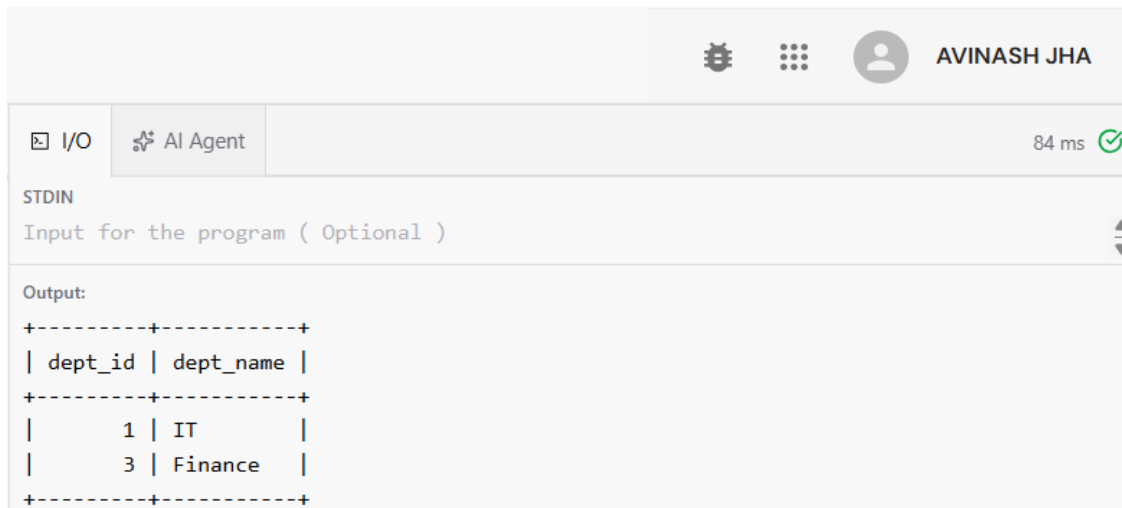
The department averages used by this query are consistent with the reference experiment's GROUP BY/HAVING results, including IT 67833.33, Finance 69166.67 and Operations 66500.00.

6. EXISTS

Find departments that have at least one employee with a salary greater than 80,000.

```
SELECT
    d.dept_id,
    d.dept_name
FROM Department d
WHERE EXISTS (
    SELECT 1
    FROM Employee e
    WHERE e.dept_id = d.dept_id
    AND e.salary > 80000
)
ORDER BY d.dept_id;
```

OUTPUT



The screenshot shows a SQL IDE interface with a user profile 'AVINASH JHA' in the top right. Below the header, there are tabs for 'I/O' and 'AI Agent'. The 'I/O' tab is active, showing 'STDIN' input and 'Output' results. The output is a table with two rows: (1, IT) and (3, Finance).

dept_id	dept_name
1	IT
3	Finance

EXISTS returns TRUE as soon as the subquery finds at least one qualifying employee for the current department.

7. SIMULATED INTERSECT

```
SELECT emp_id, emp_name, salary
FROM Employee
WHERE emp_id IN (
    SELECT emp_id
    FROM Employee
    WHERE dept_id = 1
)
AND emp_id IN (
    SELECT emp_id
    FROM Employee
    WHERE salary > 70000
)
ORDER BY emp_id;
```

AVINASH JHA

I/O
AI Agent
81 ms

STDIN

Input for the program (Optional)

Output:

emp_id	emp_name	salary
2	Vivaan	72000.00
4	Arjun	81000.00

8. SIMULATED EXCEPT

Simulate EXCEPT by selecting employees with salary greater than 70,000 while excluding employees belonging to IT.

```

SELECT emp_id, emp_name, salary
FROM Employee
WHERE emp_id IN (
    SELECT emp_id
    FROM Employee
    WHERE salary > 70000
)
AND emp_id NOT IN (
    SELECT emp_id
    FROM Employee
    WHERE dept_id = 1
)
ORDER BY emp_id;

```

AVINASH JHA

I/O
AI Agent
97 ms

STDIN

Input for the program (Optional)

Output:

emp_id	emp_name	salary
13	Rohan	75000.00
15	Nikhil	83000.00
17	Manav	71000.00
26	Dhruv	73000.00
29	Rudra	76000.00

9. EXPLAIN – EXECUTION PLAN

EXPLAIN shows how the MySQL optimizer intends to execute a SELECT statement. It can be used to inspect access type, possible indexes, selected index, estimated rows and additional operations.

A. EXPLAIN for INNER JOIN

```
EXPLAIN
SELECT
    e.emp_name,
    d.dept_name
FROM Employee e
INNER JOIN Department d
    ON e.dept_id = d.dept_id
WHERE e.salary > 70000;
```

B. EXPLAIN for Correlated Subquery

```
EXPLAIN
SELECT
    e.emp_name,
    e.salary
FROM Employee e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM Employee e2
    WHERE e2.dept_id = e.dept_id
);
```

C. EXPLAIN for EXISTS

```
EXPLAIN
SELECT d.dept_name
FROM Department d
WHERE EXISTS (
    SELECT 1
    FROM Employee e
    WHERE e.dept_id = d.dept_id
    AND e.salary > 80000
);
```

10. COMPARISON OF EXECUTION PLANS

Query	Main operation	Plan aspect to compare	Expected observation
INNER JOIN	Employee ↔ Department	Join access on dept_id	Checks how Employee and Department are accessed and joined.
Correlated subquery	Outer Employee + repeated department AVG logic	Subquery execution and rows examined	May require more work because the inner query is logically related to each outer row.
EXISTS	Department + existence test on Employee	Early existence check and filtering	Can stop searching once a qualifying Employee row is found.

For a small dataset such as the 30 employees in this experiment, the optimizer may choose table scans because the cost is low. With larger tables and suitable indexes on join/filter columns, the execution plan can change. The comparison should therefore be based on the actual EXPLAIN output produced in the lab environment.

RESULT

The Employee–Department–Project database was used to successfully demonstrate INNER JOIN, LEFT JOIN, self-join, 3-way JOIN, correlated subqueries, EXISTS, simulated INTERSECT and simulated EXCEPT. EXPLAIN statements were also prepared to compare how MySQL plans the different query forms.

CONCLUSION

Different SQL constructs can produce related results while using different execution strategies. JOIN operations combine rows from tables, correlated subqueries evaluate related values for outer rows, EXISTS tests whether matching rows exist, and subqueries can simulate set operations such as INTERSECT and EXCEPT. EXPLAIN provides information needed to study and compare these execution strategies.

EXPERIMENT - 5

AIM

To create SQL views for department salary summary and employee hierarchy, test the updatability of views, and implement a recursive CTE to display reporting chains.

OBJECTIVES

- To create a view that summarizes employee salary information department-wise.
- To create an employee hierarchy view using manager relationships.
- To test whether a view is updatable and understand the conditions affecting view updates.
- To implement a recursive CTE for displaying employee reporting chains.
- To verify the results using SELECT queries.

DATABASE USED

The queries use the CompanyDB database and the Employee–Department–Project schema from the reference experiment. The Employee table is extended with manager_id to support employee hierarchy and recursive reporting chains.

USE CompanyDB;

SCHEMA REFERENCE

The Department, Project and Employee tables remain connected through dept_id and project_id. The Employee table additionally contains manager_id, which references Employee(emp_id).

SOURCE CODE / SQL QUERIES

1. DEPARTMENT SALARY SUMMARY VIEW

Create a view showing employee count, average salary, minimum salary, maximum salary and total salary for each department.

```
CREATE OR REPLACE VIEW department_salary_summary AS
SELECT
    d.dept_id,
    d.dept_name,
    COUNT(e.emp_id) AS employee_count,
    AVG(e.salary) AS average_salary,
    MIN(e.salary) AS minimum_salary,
    MAX(e.salary) AS maximum_salary,
    SUM(e.salary) AS total_salary
FROM Department d
JOIN Employee e
    ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name;
```

Output

 AVINASH JHA

I/O AI Agent 101 ms 

STDIN

Input for the program (Optional)

Output:

dept_id	dept_name	employee_count	average_salary	minimum_salary	maximum_salary	total_salary
1	Development	6	67833.333333	58000.00	81000.00	407000.00
2	PeopleOps	6	53666.666667	47000.00	61000.00	322000.00
3	Accounts	6	69166.666667	56000.00	83000.00	415000.00
4	Sales	6	60166.666667	51000.00	70000.00	361000.00
5	Logistics	6	66500.000000	59000.00	76000.00	399000.00

2. EMPLOYEE HIERARCHY VIEW

Create a view containing each employee and the name of the employee's immediate manager.

```
CREATE OR REPLACE VIEW employee_hierarchy AS
SELECT
    e.emp_id,
    e.emp_name,
    e.dept_id,
    d.dept_name,
    e.manager_id,
    m.emp_name AS manager_name,
    e.salary
FROM Employee e
JOIN Department d
    ON e.dept_id = d.dept_id
LEFT JOIN Employee m
    ON e.manager_id = m.emp_id;
```

Output

I/O

AI Agent

84 ms

AVINASH JHA

STDIN

Input for the program (Optional)

Output:

emp_id	emp_name	dept_id	dept_name	manager_id	manager_name	salary
1	Ishaan	1	Development	NULL	NULL	65000.00
2	Reyansh	1	Development	1	Ishaan	72000.00
3	Adit	1	Development	2	Reyansh	58000.00
4	Aarush	1	Development	3	Adit	81000.00
5	Raghav	1	Development	4	Aarush	69000.00
6	Vihaan	1	Development	5	Raghav	62000.00
7	Anika	2	PeopleOps	NULL	NULL	52000.00
8	Kavya	2	PeopleOps	7	Anika	48000.00
9	Ira	2	PeopleOps	8	Kavya	61000.00
10	Naina	2	PeopleOps	9	Ira	55000.00
11	Aditi	2	PeopleOps	10	Naina	47000.00
12	Saanvi	2	PeopleOps	11	Aditi	59000.00
13	Harsh	3	Accounts	NULL	NULL	75000.00
14	Mohit	3	Accounts	13	Harsh	68000.00
15	Ritvik	3	Accounts	14	Mohit	83000.00
16	Ayush	3	Accounts	15	Ritvik	62000.00
17	Kunal	3	Accounts	16	Ayush	71000.00
18	Varun	3	Accounts	17	Kunal	56000.00
19	Mehak	4	Sales	NULL	NULL	54000.00
20	Pihu	4	Sales	19	Mehak	63000.00
21	Shreya	4	Sales	20	Pihu	57000.00
22	Tara	4	Sales	21	Shreya	66000.00
23	Mahi	4	Sales	22	Tara	51000.00
24	Anvi	4	Sales	23	Mahi	70000.00
25	Rajat	5	Logistics	NULL	NULL	60000.00
26	Yuvraj	5	Logistics	25	Rajat	73000.00
27	Arnav	5	Logistics	26	Yuvraj	67000.00
28	Laksh	5	Logistics	27	Arnav	59000.00
29	Devansh	5	Logistics	28	Laksh	76000.00
30	Om	5	Logistics	29	Devansh	64000.00

The hierarchy view shows the immediate reporting relationship. A department head has NULL in manager_id and manager_name.

3. TEST UPDATABILITY OF VIEWS

Test an update on a simple employee hierarchy view and then test an update on the grouped department salary summary view.

```
UPDATE employee_hierarchy
SET salary = salary + 1000
WHERE emp_id = 6;
```

```
SELECT emp_id, emp_name, salary
FROM Employee
WHERE emp_id = 6;
```

```
UPDATE department_salary_summary
SET average_salary = average_salary + 1000
WHERE dept_id = 1;
```

Output

The first UPDATE is expected to fail because the employee_hierarchy view contains joins and does not directly expose a single base-table row for every selected column. The grouped department_salary_summary view is also not directly updatable because it contains GROUP BY and aggregate functions. The base Employee table should be updated instead.

4. RECURSIVE CTE – REPORTING CHAIN

Display the reporting chain beginning with department heads and recursively follow manager relationships.

```
WITH RECURSIVE reporting_chain AS (
    SELECT
        e.emp_id,
        e.emp_name,
        e.manager_id,
        1 AS level,
        CAST(e.emp_name AS CHAR(500)) AS reporting_path
    FROM Employee e
    WHERE e.manager_id IS NULL

    UNION ALL

    SELECT
        e.emp_id,
        e.emp_name,
        e.manager_id,
        rc.level + 1,
        CONCAT(rc.reporting_path, ' -> ', e.emp_name)
    FROM Employee e
    JOIN reporting_chain rc
        ON e.manager_id = rc.emp_id
)
SELECT emp_id, emp_name, manager_id, level, reporting_path
FROM reporting_chain
ORDER BY reporting_path;
```

Output

I/O

AI Agent

87 ms

AVINASH JHA

STDIN

Input for the program (Optional)

Output:

emp_id	emp_name	manager_id	level	reporting_path
7	Anika	NULL	1	Anika
8	Kavya	7	2	Anika -> Kavya
9	Ira	8	3	Anika -> Kavya -> Ira
10	Naina	9	4	Anika -> Kavya -> Ira -> Naina
11	Aditi	10	5	Anika -> Kavya -> Ira -> Naina -> Aditi
12	Saanvi	11	6	Anika -> Kavya -> Ira -> Naina -> Aditi -> Saanvi
13	Harsh	NULL	1	Harsh
14	Mohit	13	2	Harsh -> Mohit
15	Ritvik	14	3	Harsh -> Mohit -> Ritvik
16	Ayush	15	4	Harsh -> Mohit -> Ritvik -> Ayush
17	Kunal	16	5	Harsh -> Mohit -> Ritvik -> Ayush -> Kunal
18	Varun	17	6	Harsh -> Mohit -> Ritvik -> Ayush -> Kunal -> Varun
1	Ishaan	NULL	1	Ishaan
2	Reyansh	1	2	Ishaan -> Reyansh
3	Adit	2	3	Ishaan -> Reyansh -> Adit
4	Aarush	3	4	Ishaan -> Reyansh -> Adit -> Aarush
5	Raghav	4	5	Ishaan -> Reyansh -> Adit -> Aarush -> Raghav
6	Vihaan	5	6	Ishaan -> Reyansh -> Adit -> Aarush -> Raghav -> Vihaan
19	Mehak	NULL	1	Mehak
20	Pihu	19	2	Mehak -> Pihu
21	Shreya	20	3	Mehak -> Pihu -> Shreya
22	Tara	21	4	Mehak -> Pihu -> Shreya -> Tara
23	Mahi	22	5	Mehak -> Pihu -> Shreya -> Tara -> Mahi
24	Anvi	23	6	Mehak -> Pihu -> Shreya -> Tara -> Mahi -> Anvi
25	Rajat	NULL	1	Rajat
26	Yuvraj	25	2	Rajat -> Yuvraj
27	Arnav	26	3	Rajat -> Yuvraj -> Arnav
28	Laksh	27	4	Rajat -> Yuvraj -> Arnav -> Laksh
29	Devansh	28	5	Rajat -> Yuvraj -> Arnav -> Laksh -> Devansh
30	Om	29	6	Rajat -> Yuvraj -> Arnav -> Laksh -> Devansh -> Om

RESULT

Views were created for department salary summaries and employee hierarchy. View update behavior was tested, and a recursive CTE was used successfully to display reporting chains.

CONCLUSION

SQL views provide reusable logical representations of data, while recursive CTEs can traverse self-referencing employee relationships to display hierarchical reporting structures.

EXPERIMENT - 6

AIM

To create a stored procedure `transfer_employee(emp_id, new_dept_id)` with validation and error handling, implement triggers for salary validation and audit logging, and test important edge cases.

OBJECTIVES

- To create a stored procedure for transferring an employee between departments.
- To validate employee and department identifiers before performing a transfer.
- To use transaction control and error handling inside a stored procedure.
- To validate salary values using a trigger before INSERT or UPDATE.
- To maintain an audit log whenever an employee salary is changed.
- To test successful and invalid transfer and salary-update cases.

DATABASE USED

The queries use the CompanyDB database and the Employee–Department–Project schema. Department names and employee names have been changed from the reference data; IDs remain unchanged so the existing relationships continue to work.

USE CompanyDB;

SOURCE CODE / SQL QUERIES

1. SALARY AUDIT TABLE

Create an audit table to store old and new salary values whenever an employee salary changes.

```
CREATE TABLE IF NOT EXISTS Employee_Salary_Audit (  
    audit_id INT AUTO_INCREMENT PRIMARY KEY,  
    emp_id INT NOT NULL,  
    old_salary DECIMAL(10,2),  
    new_salary DECIMAL(10,2),  
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    action VARCHAR(30)  
);
```

2. SALARY VALIDATION TRIGGER

Reject salary values outside the allowed range of 30,000 to 1,50,000.

```
DELIMITER //
CREATE TRIGGER validate_employee_salary
BEFORE INSERT ON Employee
FOR EACH ROW
BEGIN
    IF NEW.salary < 30000 OR NEW.salary > 150000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary must be between 30000 and 150000';
    END IF;
END//
DELIMITER ;
```

3. SALARY UPDATE VALIDATION TRIGGER

Validate salary changes before an existing Employee row is updated.

```
UPDATE Employee
SET salary = 20000
WHERE emp_id = 6;
```

Valid Update:

```
UPDATE Employee
SET salary = 85000
WHERE emp_id = 6;
```

 AVINASH JHA



Error

STDIN

Input for the program (Optional)

Output:

ERROR 1644 (45000) at line 118: Salary must be between 30000 and 150000

 Debug this error

4. SALARY AUDIT TRIGGER

Store the old and new salary values after a successful salary update.

Trigger code:

```
DELIMITER //
```

```
CREATE TRIGGER audit_salary_update
```

```
AFTER UPDATE ON Employee
```

```
FOR EACH ROW
```

```
BEGIN
```

```
IF OLD.salary <> NEW.salary THEN
```

```
INSERT INTO Employee_Salary_Audit
```

```
(emp_id, old_salary, new_salary, action)
```

```
VALUES
```

```
(NEW.emp_id, OLD.salary, NEW.salary, 'SALARY UPDATE');
```

```
END IF;
```

```
END//
```

```
DELIMITER;
```

CODE:

```
DELIMITER //
```

```
-- Test salary update
```

```
UPDATE Employee
```

```
SET salary = 51000
```

```
WHERE emp_id = 31;
```



```
-- Display salary audit record
```

```
SELECT audit_id, emp_id, old_salary, new_salary,
```

```
       changed_at, action
```

```
FROM Employee_Salary_Audit
```

```
ORDER BY audit_id;
```

```
DELIMITER ;
```

Output

 AVINASH JHA

I/O

 AI Agent

121 ms 

STDIN

Input for the program (Optional)

Output:

audit_id	emp_id	old_salary	new_salary	changed_at	action
1	31	50000.00	51000.00	2026-09-22 11:32:30	SALARY UPDATE

5. STORED PROCEDURE – TRANSFER EMPLOYEE

```
DELIMITER //
CREATE PROCEDURE transfer_employee(
    IN p_emp_id INT,
    IN p_new_dept_id INT
)
BEGIN
    DECLARE v_emp_count INT DEFAULT 0;
    DECLARE v_dept_count INT DEFAULT 0;
    DECLARE v_old_dept_id INT;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;
    SELECT COUNT(*) INTO v_emp_count
    FROM Employee
    WHERE emp_id = p_emp_id;

    IF v_emp_count = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Employee does not exist';
    END IF;

    SELECT COUNT(*) INTO v_dept_count
    FROM Department
    WHERE dept_id = p_new_dept_id;

    IF v_dept_count = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Department does not exist';
    END IF;

    SELECT dept_id INTO v_old_dept_id
    FROM Employee
    WHERE emp_id = p_emp_id;

    IF v_old_dept_id = p_new_dept_id THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Employee is already in this department';
    END IF;

    UPDATE Employee
    SET dept_id = p_new_dept_id
    WHERE emp_id = p_emp_id;

    COMMIT;
END//
DELIMITER ;
```

6. TEST SUCCESSFUL TRANSFER

Transfer employee 6 from Development to Sales.

```
CALL transfer_employee(6, 4);
```

```
SELECT e.emp_id, e.emp_name,  
       d.dept_name  
FROM Employee e  
JOIN Department d ON e.dept_id = d.dept_id  
WHERE e.emp_id = 6;
```

Output

 AVINASH JHA

 I/O  AI Agent 95 ms 

STDIN

Input for the program (Optional)

Output:

emp_id	emp_name	dept_name
6	Vihaan	Sales

The procedure validates the employee and department, updates the department and commits the transaction.

7. TEST EDGE CASES

```
-- Invalid employee  
CALL transfer_employee(99, 4);
```

```
-- Invalid department  
CALL transfer_employee(6, 99);
```

```
-- Same department  
CALL transfer_employee(6, 4);
```

Output

queries.sql

MySQL Run

AI Agent

Error

```
154 BEGIN
175
176 SELECT COUNT(*) INTO v_dept_count
177 FROM Department
178 WHERE dept_id = p_new_dept_id;
179
180 IF v_dept_count = 0 THEN
181     SIGNAL SQLSTATE '45000'
182     SET MESSAGE_TEXT = 'Department does not exist';
183 END IF;
184
185 SELECT dept_id INTO v_old_dept_id
186 FROM Employee
187 WHERE emp_id = p_emp_id;
188
189 IF v_old_dept_id = p_new_dept_id THEN
190     SIGNAL SQLSTATE '45000'
191     SET MESSAGE_TEXT = 'Employee is already in this department';
192 END IF;
193
194 UPDATE Employee
195 SET dept_id = p_new_dept_id
196 WHERE emp_id = p_emp_id;
197
198 COMMIT;
199 END//
200 DELIMITER ;
201
202 CALL transfer_employee(6, 4);
203
204 -- Same department
205 CALL transfer_employee(6, 4);
206
207
208 SELECT e.emp_id, e.emp_name,
209        d.dept_name
210 FROM Employee e
211 JOIN Department d ON e.dept_id = d.dept_id
212 WHERE e.emp_id = 6;
213
214
215
216
```

STDM
Input for the program (Optional)

Output
ERROR 1644 (45000) at line 207: Employee is already in this department
Debug this error

queries.sql

MySQL Run

AI Agent

Error

```
154 BEGIN
175
176 SELECT COUNT(*) INTO v_dept_count
177 FROM Department
178 WHERE dept_id = p_new_dept_id;
179
180 IF v_dept_count = 0 THEN
181     SIGNAL SQLSTATE '45000'
182     SET MESSAGE_TEXT = 'Department does not exist';
183 END IF;
184
185 SELECT dept_id INTO v_old_dept_id
186 FROM Employee
187 WHERE emp_id = p_emp_id;
188
189 IF v_old_dept_id = p_new_dept_id THEN
190     SIGNAL SQLSTATE '45000'
191     SET MESSAGE_TEXT = 'Employee is already in this department';
192 END IF;
193
194 UPDATE Employee
195 SET dept_id = p_new_dept_id
196 WHERE emp_id = p_emp_id;
197
198 COMMIT;
199 END//
200 DELIMITER ;
201
202 |
203
204 -- Invalid employee
205 CALL transfer_employee(99, 4);
206
207
208
209
210 SELECT e.emp_id, e.emp_name,
211        d.dept_name
212 FROM Employee e
213 JOIN Department d ON e.dept_id = d.dept_id
214 WHERE e.emp_id = 6;
215
216
```

STDM
Input for the program (Optional)

Output
ERROR 1644 (45000) at line 205: Employee does not exist
Debug this error

queries.sql x +MySQL Run

```
154 BEGIN
155
156 SELECT COUNT(*) INTO v_dept_count
157 FROM Department
158 WHERE dept_id = p_new_dept_id;
159
160 IF v_dept_count = 0 THEN
161     SIGNAL SQLSTATE '45000'
162     SET MESSAGE_TEXT = 'Department does not exist';
163 END IF;
164
165 SELECT dept_id INTO v_old_dept_id
166 FROM Employee
167 WHERE emp_id = p_emp_id;
168
169 IF v_old_dept_id = p_new_dept_id THEN
170     SIGNAL SQLSTATE '45000'
171     SET MESSAGE_TEXT = 'Employee is already in this department';
172 END IF;
173
174 UPDATE Employee
175 SET dept_id = p_new_dept_id
176 WHERE emp_id = p_emp_id;
177
178 COMMIT;
179
180 END//
181 DELIMITER ;
182
183 -- Invalid department
184 CALL transfer_employee(6, 99);
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210 SELECT e.emp_id, e.emp_name,
211        d.dept_name
212 FROM Employee e
213 JOIN Department d ON e.dept_id = d.dept_id
214 WHERE e.emp_id = 6;
215
216
```

STDIN

Input for the program (Optional)

Output

ERROR 1644 (45000) at line 204: Department does not exist

Debug this error

8. TEST SALARY VALIDATION

Attempt to insert or update a salary outside the allowed range.

```
UPDATE Employee
SET salary = 20000
WHERE emp_id = 6;
```

```
UPDATE Employee
SET salary = 85000
WHERE emp_id = 6;
```

Expected Output

ERROR 1644 (45000): Salary must be between 30000 and 150000

Output

queries.sql x +MySQL Run

```
154 BEGIN
155
156 SELECT dept_id INTO v_old_dept_id
157 FROM Employee
158 WHERE emp_id = p_emp_id;
159
160 IF v_old_dept_id = p_new_dept_id THEN
161     SIGNAL SQLSTATE '45000'
162     SET MESSAGE_TEXT = 'Employee is already in this department';
163 END IF;
164
165 UPDATE Employee
166 SET dept_id = p_new_dept_id
167 WHERE emp_id = p_emp_id;
168
169 COMMIT;
170
171 END//
172 DELIMITER ;
173
174 DROP TRIGGER IF EXISTS validate_employee_salary_update;
175
176 DELIMITER //
177
178 CREATE TRIGGER validate_employee_salary_update
179 BEFORE UPDATE ON Employee
180 FOR EACH ROW
181 BEGIN
182     IF NEW.salary < 30000 OR NEW.salary > 150000 THEN
183         SIGNAL SQLSTATE '45000'
184         SET MESSAGE_TEXT = 'Salary must be between 30000 and 150000';
185     END IF;
186 END//
187
188 DELIMITER ;
189
190 UPDATE Employee
191 SET salary = 20000
192 WHERE emp_id = 6;
```

STDIN

Input for the program (Optional)

Output

ERROR 1644 (45000) at line 218: Salary must be between 30000 and 150000

Debug this error

9. TEST SALARY AUDIT LOG

Display the audit entries created by successful salary changes.

```
SELECT audit_id, emp_id, old_salary, new_salary,  
       changed_at, action  
FROM Employee_Salary_Audit  
ORDER BY audit_id;
```

Output

   AVINASH JHA

I/O

AI Agent

98 ms 

STDIN

Input for the program (Optional)

Output:

audit_id	emp_id	old_salary	new_salary	changed_at	action
1	31	50000.00	51000.00	2026-09-22 12:38:17	SALARY UPDATE

RESULT

The transfer_employee procedure was created with validation, transaction control and error handling. Salary validation and audit triggers were implemented and edge cases were tested.

CONCLUSION

Stored procedures centralize controlled database operations, while triggers automatically enforce validation rules and maintain an audit trail for important changes.